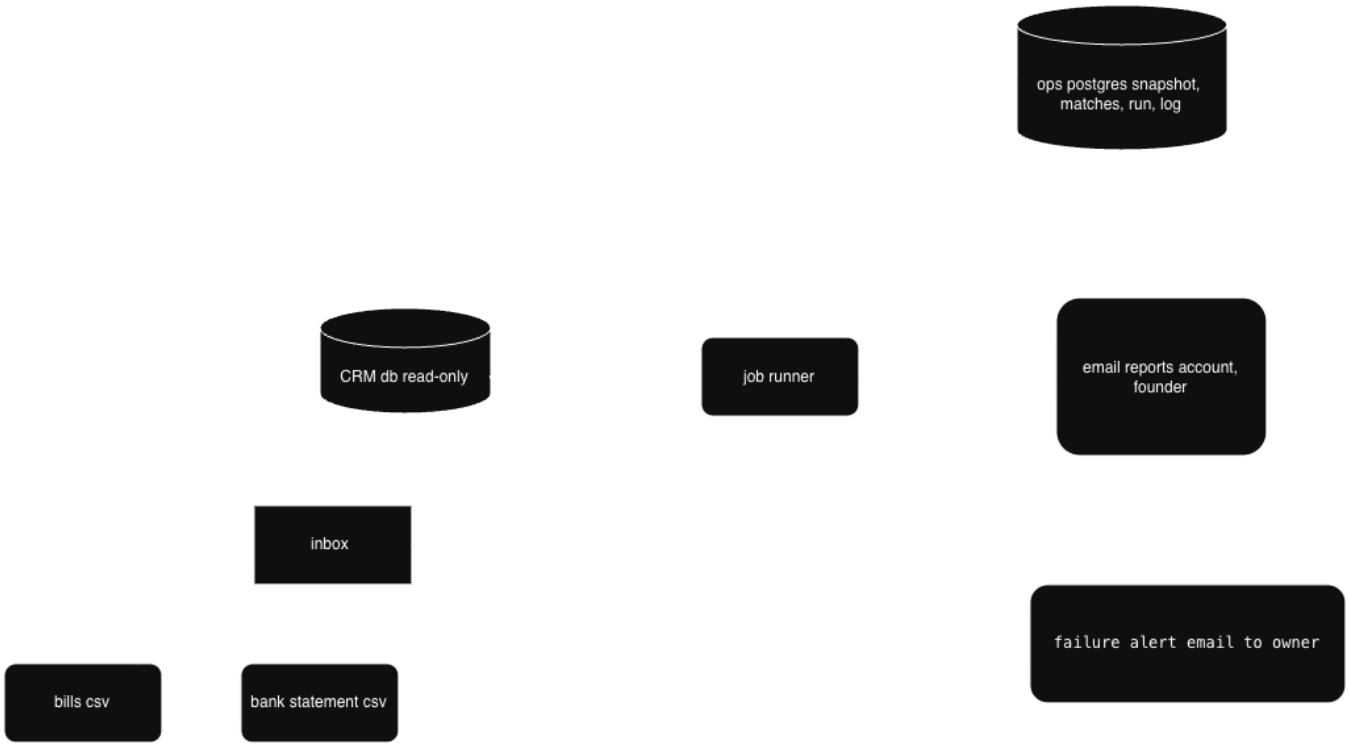
One small Linux server running five scheduled Python jobs against the CRM's read-only database mirror, writing to its own small Postgres database, and emailing results. That is the whole design, and the rest of this document explains why it doesn't need more.

Inputs. Three sources feed the design: (1) a read-only mirror of the CRM database, (2) a bills export from the CRM, and (3) bank statements. The business earns from recurring bookings plus add-on upsells, and the CRM is about 12 years old.

1. Architecture

Five cron-triggered batch jobs on one server, one small Postgres database for the jobs' own state, email as the only output channel. No queue, no containers orchestrator, no web app.



Data flows left to right. The CRM is never written to.

Components

Component	What it is	Why this and not more
Data sources	Read-only connection to the CRM mirror; a drop folder (SFTP or a shared drive synced to the server) for the bills export and bank statements	The client already produces these. Nothing new for them to operate
Job runner	Plain Python scripts, one per automation, triggered by cron	Five jobs on fixed schedules is exactly what cron is for. Airflow or Prefect would add a scheduler, a UI and a metadata DB to babysit five jobs
Ops database	Postgres on the same server	Holds daily snapshots, the change log, reconciliation matches and the run log. Needed because the mirror can't be written to and has no history
Output	Emailed reports: CSV or XLSX attachment plus a short summary in the body	Accounts and the founder already live in email. A dashboard is something someone has to remember to open
Secrets	One `.env` file readable only by the job user	A secrets manager is justified by many services and many people; this has one of each

Triggers

Job	Trigger	Waits on
Snapshot + change log	Daily, 05:00	Mirror refresh finished (checked, see below)
Outstanding bills	Daily, 06:00	Snapshot job succeeded

Job	Trigger	Waits on
Reconciliation	When a new statement file lands in the inbox (checked every 15 min), plus a daily 07:00 sweep	Snapshot job succeeded that day
Billing report	Weekly, Monday 07:30, plus a month-end run	Snapshot + bills export present
Founder report	Weekly, Monday 08:00	Reconciliation done for the prior week

Every job reads from the ops database's snapshot, not live from the mirror. That means all five reports agree with each other for a given day, which matters more to accounts than freshness.

On failure

- **Pre-flight checks before any job runs:** mirror is reachable; its newest `updated_at` (or max ID) moved since yesterday, so we know it actually refreshed; row counts in key tables are within ±20% of yesterday; expected input files exist and parse. Any failure stops that job.
- **Fail loudly, never silently send wrong numbers.** A failed job sends one alert email to a named owner with the step that failed and the error. It does not send a partial report. A report that's late is recoverable; one with wrong totals erodes trust in all of them.
- **Retry once** after 30 minutes for connection errors only. Data-validation failures are not retried; retrying won't fix bad data.
- **Idempotent runs.** Each job writes under a run date, so re-running a day replaces that day's output rather than doubling it.
- **Dead-man check.** If a job hasn't recorded success by its expected time plus 2 hours, a tiny check job emails the owner. This catches the case where cron itself or the server stopped.
- **Backups:** the provider's daily server backup plus a nightly `pg_dump` of the ops database copied off-server. The ops database is the only thing here that can't be rebuilt from the source (snapshot history is the change log).

Why this simple

The workload is five jobs, each running for seconds to a few minutes, a handful of times a day, for maybe three readers. There is no user-facing latency, no concurrency and no scaling curve. Every extra component (queue, container orchestration, separate DB host, BI tool) is something the client pays for monthly and someone has to understand when it breaks, and a small business very likely has no one in that role after handover. **I deliberately chose cron over a workflow orchestrator and email over a dashboard.** The point where I'd revisit this: if jobs grow past roughly 15, or start depending on each other in more than a straight line, a lightweight orchestrator starts earning its keep.

2. Server & cost

Recommendation: one **DigitalOcean Basic Droplet, 2 GiB RAM / 1 vCPU / 50 GiB SSD, in the Bangalore (BLR1) region: about ₹1,152/month**, plus daily backups at 30% of the droplet price (about ₹346), for **about ₹1,498/month** total, before 18% GST. DigitalOcean bills in USD ($12.00 + $3.60 = $15.60), so the rupee figures use ₹96.0 per USD as of 25 Sep 2026 and will move with the exchange rate. Source: DigitalOcean Droplet pricing, checked 25 Sep 2026.

Option	Spec	Price/month (approx., excl. GST)	Verdict
Basic 1 GiB	1 vCPU, 25 GiB SSD	₹576 ($6)	Too small: Postgres plus a Python process loading a day's snapshot will hit swap
Basic 2 GiB	**1 vCPU, 50 GiB SSD**	**₹1,152 ($12)**	**Chosen**
Basic 4 GiB	2 vCPU, 80 GiB SSD	₹2,304 ($24)	Upgrade path if snapshot diffs outgrow memory

Why this size. The jobs run one at a time on a staggered schedule, so a single vCPU is never contended; nothing is serving live traffic. Memory is the real constraint, because Postgres and a Python job share the box. 2 GiB is enough if the heavy work (snapshot diffs, ageing buckets) runs as SQL inside Postgres rather than pulling whole tables into pandas. 50 GiB of disk holds years of daily snapshots for a business this size, especially if old snapshots are compacted to only changed rows.

Why not bigger. Paying for 4 GiB up front buys headroom the workload probably never uses. A droplet resize to the next tier takes minutes, so it's cheaper to start small and move up if memory is tight.

Why not smaller. At 1 GiB, one bad query plan on a large table could push the server into swap on the morning the accounts team needs the report.

Judgment call. The CRM's table sizes are unknown. If the bookings or bills tables run to millions of rows, I'd start on the 4 GiB tier instead. I picked DigitalOcean for its flat, published pricing, an Indian data centre (useful for data residency, see A14), and a console a non-technical owner can manage after handover.

Also needed, not on the server: a transactional email service for sending reports. Most have a free tier that covers a few hundred emails a month, which is far more than this sends.

3. Assumptions

Each of these is a guess I've made to design something concrete. The last column says what changes if it's wrong.

#	Assumption	If it's wrong
A1	The DB mirror refreshes **once a day, overnight**, as a full copy	If it's near-real-time replication, binlog-based change capture becomes possible (see Change log) and jobs could run later in the morning
A2	The mirror has a detectable refresh marker (max `updated_at`, max ID, or a refresh timestamp)	Pre-flight can't tell a stale mirror from a quiet day; I'd need the client's IT to expose refresh status
A3	A **stable unique customer ID** exists and is used on bookings and bills	If customers are keyed by name or email, duplicates are likely and every report needs a dedup step first
A4	Bills have a unique bill/invoice number that is printed on the invoice the customer sees	Without it, bank references can't be tied to bills and reconciliation falls back to amount + name only, with far lower confidence

#	Assumption	If it's wrong
A5	The **bills export does not exactly match** the raw bills table (it may apply filters, rounding, or exclude voided bills)	I've assumed we reconcile the two on day one and pick one as the source of truth. If they match, that step is a no-op
A6	The bank statement arrives as a **file (CSV or OFX) dropped manually**, not via a bank API	If an API or aggregator feed exists, reconciliation can run daily without anyone uploading anything
A7	One business bank account, one currency	Multiple accounts or FX means matching per account and a currency-conversion tolerance
A8	Payments arrive by bank transfer with a free-text reference field; card payments settle in batched payouts	If most payments are card via a processor, reconciliation should match against processor payout reports, not the bank line
A9	The CRM keeps **no change history** and has no reliable audit table	If an audit table exists, the change log job just reads it and the snapshot-diff machinery is unnecessary
A10	Bills carry an issue date and a due date (or a standard payment term I can apply)	Ageing needs a due date; with only issue dates I'd age from issue date + a term the client confirms
A11	Add-ons are **separate line items or products** in the CRM, distinguishable from the core booking	If add-ons are baked into a single booking price, the upsell metrics in the founder report can't be computed
A12	Recurring bookings are identifiable (a subscription/plan record, or a repeat flag)	If not, I'd infer recurrence from repeat bookings by the same customer, which is fuzzier
A13	Reports go to a small, fixed set of people (accounts team, founder) by email	Many recipients or role-based access pushes toward a small web view with login
A14	Customer data is personal data under India's Digital Personal Data Protection Act, 2023	The server should be in the client's region, with encrypted backups and a retention policy for snapshots
A15	Data volume: tens of thousands of customers, not millions	Server size in section 2 changes

#	Assumption	If it's wrong
A16	Someone on the client side can own alerts after handover	If no one can, alerts should go to a support contract, and the design should be even simpler

4. Per-automation approach

4.1 Billing report to accounts

Schedule. Weekly on Monday morning covering the previous week, plus a month-end run on the first working day covering the full month. Accounts usually closes monthly; the weekly run is there so surprises show up before month-end, not at it.

Contents.

- Summary in the email body: bills issued (count and total), payments received, credit notes/refunds, net change in amount owed.
- Attached spreadsheet: one row per bill issued in the period, with customer ID, customer name, bill number, issue date, due date, core booking amount, add-on amount, tax, total, status.
- A **tie-out line**: total from the raw DB vs total from the bills export, and the difference.

When data looks wrong. The job runs checks before sending:

- Export total and DB total differ by more than a set tolerance (e.g. ₹100 or 0.1%).
- Bills with a zero or negative total that are not credit notes.
- Duplicate bill numbers.
- Bills referencing a customer ID that doesn't exist.
- Period total more than ±40% off the trailing 8-week average.

The first three block the report, since they mean the numbers themselves can't be trusted, and an alert goes to the owner. The last two send the report with a clearly marked **Needs review** section at the top listing the specific rows. Judgment call: the thresholds above are placeholders to be tuned with accounts after a few weeks of real runs.

4.2 Change log

The honest starting point: a 12-year-old CRM almost certainly doesn't record who changed what and when. At best it has an `updated_at` column, which tells you a row changed but not what the previous value was.

Options considered.

Approach	How it works	Catches	Misses / cost
Binlog / CDC (e.g. reading MySQL binlog with Debezium)	Streams every insert, update, delete from the primary database's replication log	Every change, with before/after values and exact time	Needs access to the **primary** database's log and server config changes. We only have a mirror. Adds a long-running streaming service to operate
`updated_at` polling	Select rows where `updated_at` > last run	Which rows changed	No before-values; misses deletes; only as good as the app's discipline in setting the column, which in old systems is often patchy
Daily snapshot diff (chosen)	Copy key tables into the ops DB each day; compare today vs yesterday row by row on primary key	Inserts, updates (with old and new value per field) and deletes	Only sees **end-of-day state**. No "who"

Why snapshot diff. It works with exactly the access we have (a read-only mirror), needs no change to the client's system, and runs as one more nightly batch step. Binlog would be more complete, but it requires access we don't have and a service that must run all day. That is too much for a report someone reads once a day.

Limitations, stated plainly in the report footer:

- **Intra-day changes are invisible.** If a price goes from ₹1,000 to ₹800 and back to ₹1,000 within a day, the log shows nothing.
- **No author.** We can say *what* changed, not *who* changed it, unless the CRM stores a last-modified-by column.
- **The log starts on day one.** No history before the first snapshot.
- **Accuracy depends on the mirror refresh.** If the mirror skips a day, two days of changes collapse into one diff. The pre-flight check flags this.

What goes in the log. Not every field. I'd limit it to fields that matter to money or customer status: prices, bill amounts, bill status, discounts, customer contact and bank details, booking cancellations, and deleted bills. Deleted or edited bills after issue get top billing in the email, since that's what an accountant or auditor actually wants to know about.

4.3 Outstanding bills

Calculation. For each bill: outstanding = bill total − confirmed payments allocated to it − credit notes against it. Only payments confirmed by reconciliation (4.4) count; payments that are merely flagged don't reduce the balance until someone accepts them. Days overdue = report date − due date (A10).

Ageing buckets: not yet due, 1–30, 31–60, 61–90, 90+ days overdue. These are the standard accounting buckets, so accounts can use them as-is.

How it's shown.

- Email body: total outstanding, total overdue, the amount in each bucket, and the change since last week per bucket. The week-on-week change is the useful signal; a static total isn't.
- Attachment: one row per customer with a balance, sorted by oldest overdue amount first, with bills listed under each customer. Customer-level matters because one customer with five late bills is one phone call, not five.
- A short "newly 60+" list: customers who crossed into 61–90 since the last report. This is the list someone should act on this week.

Who sees it. Accounts gets the full daily file. The founder gets only the bucket totals, inside the weekly founder report. Judgment call: I'd make it daily for accounts only if they chase payments daily; otherwise weekly is enough and less noise.

Caveat. Outstanding balances are only as accurate as reconciliation. If a customer paid but the payment is stuck in the low-confidence queue, they'll show as overdue. The report marks such customers "payment pending review" so no one chases a customer who has

already paid.

4.4 Bank statement vs CRM reconciliation

Principle: the system proposes, a person confirms anything uncertain. Only exact, unambiguous matches are confirmed automatically. Everything else is a suggestion in a review queue with the reason it wasn't confident. A wrong auto-match is worse than no match: it marks an unpaid bill as paid, and nobody chases it.

Scope. Match every **incoming** bank line to one or more open bills. Also run the reverse check: bills the CRM marks as paid with no matching bank line. That reverse case (the CRM says paid, the bank says no) is the one most likely to hide an actual problem.

Pre-processing

- Normalise bank lines: amount to pennies as an integer, date, and a cleaned reference (uppercase, punctuation and bank prefixes stripped).
- Deduplicate statement lines across overlapping files using date + amount + reference + running balance, because people will upload the same week twice.
- Store every line in the ops DB with its source file, so each match can be traced back to the statement.

Matching order

The order matters. Each tier runs only on lines the previous tier didn't settle.

Tier	Signal	Outcome	Why it's at this position
1	**Bill number found in the bank reference** AND amount equals that bill's outstanding balance exactly	Auto-confirm	The bill number is the only signal the customer deliberately attached to *this* payment. Amount alone is shared by many customers; names are messy. When both agree, the chance of a wrong match is very low
2	Bill number in reference, amount differs	Suggest; reason: partial or overpayment	The customer identified the bill but the amount says something else is going on

Tier	Signal	Outcome	Why it's at this position
3	Customer ID or a known payer reference in the reference, amount equals one open bill or an exact sum of that customer's open bills	Suggest (high confidence)	Right customer, but which bills it covers is an inference
4	Amount equals exactly one open bill in the whole book, within ±7 days of due date, and payer name is a fuzzy match to that customer	Suggest (medium confidence)	Three weak signals agreeing. Still an inference
5	Anything else	Unmatched queue	No basis to guess

Judgment call: I auto-confirm only tier 1. Tier 3 is tempting to auto-confirm once the client trusts the system; I'd revisit that after reviewing a couple of months of how often reviewers override tier 3.

Learning without ML. When a reviewer confirms that bank payer name "J SMITH LTD" belongs to customer 4417, store that alias. Next time, the alias acts like a customer ID in tier 3. This is a lookup table, not a model, and it's what makes the queue shrink over time. I chose this over a trained matcher deliberately: it's explainable to an accountant, and the volume won't support training a model anyway.

Edge cases

Case	What happens
Partial payment (₹1,500 against a ₹3,000 bill)	Tier 2. Suggested allocation of ₹1,500 to that bill, leaving ₹1,500 outstanding. Never auto-confirmed, because a short payment may also mean a disputed line or a wrong bill

Case	What happens
One payment covers several bills (₹4,500 = three ₹1,500 bills)	If the reference lists several bill numbers, allocate to those. Otherwise, search that customer's open bills for combinations summing exactly to the payment, oldest first. If exactly one combination works, suggest it. If several work (e.g. 3 of 5 identical monthly bills), suggest the oldest, flagged. Search is bounded to one customer and ~10 open bills, so it stays cheap
Same amount, different customers (two customers each owe ₹999 on the same monthly plan)	This is the most common false-match trap for a recurring-bookings business, since many bills share identical amounts. Amount alone never produces a match. With no reference or name to break the tie, it goes to the unmatched queue listing all candidates
Name mismatch (paid by a spouse, a company, or "MR J A SMITH" vs "John Smith")	Fuzzy name matching only ever *supports* a match, never creates one. Confirmed aliases (above) handle the recurring cases
Refunds / outgoing payments	Outgoing lines are matched against CRM credit notes/refunds by amount and customer. A refund with no credit note in the CRM is flagged: money left without a record
Bounced / reversed payments	A reversal line that exactly offsets a prior matched payment un-matches it and reopens the bill, and is flagged for accounts
Overpayment	Allocate up to the bill total; surplus held as customer credit, flagged. Never silently applied to the next bill
Bank fees / card processor payouts	A single payout covering many card payments cannot be matched line-to-line from the bank statement. Needs the processor's payout report (A8). Out of scope until that's confirmed
Payment before bill exists (prepayment for a booking)	Held in unmatched; re-tried on each run for 30 days as new bills appear

Case	What happens
CRM says paid, no bank line	Reported separately. Could be cash, a different account, or a data-entry error

Low-confidence matches

They are **never confirmed automatically, whatever the score.** They go to a review file (spreadsheet) sent to accounts with: the bank line, the suggested bill(s), the tier, and a plain-English reason ("amount matches but payer name differs"). Accounts marks each row accept / reject / reassign and drops the file back in the inbox. The next run applies those decisions and records who made them. Until confirmed, a suggested match does not reduce the outstanding balance (4.3).

Judgment call: a spreadsheet round-trip is clunky, and if the queue is more than ~20 lines a week, a single-page review screen would be worth building. I'd start with the spreadsheet and build the screen only if volume proves it's needed.

Output: matched count and value, suggested count and value, unmatched count and value, and the reverse-check list. A match rate trending down is itself a signal: usually customers have stopped using the bill number as their reference, which is fixable by changing the invoice template.

4.5 Founder report

Five numbers, weekly, in the body of one email, each with last week and the 8-week trend. No dashboard. The business makes money two ways, from customers who keep booking and from what they add on, so every metric below measures one of those.

#	Metric	Definition	Business lever	Decision it changes
1	**Active recurring customers, net change**	Customers with a recurring booking at week end, split into new, lost and net	Recurring base	Whether to spend on acquisition or retention this month. Growth masked by high churn looks fine in revenue but isn't

#	Metric	Definition	Business lever	Decision it changes
2	**Recurring churn rate**	Recurring customers who cancelled or lapsed this period ÷ active at start; with the top cancellation reasons if the CRM records them	Recurring base	Whether to act on service or pricing. A rise concentrated in one service or area points to a specific fix
3	**Add-on attach rate**	% of bookings this period with at least one add-on	Upsell	Whether upsell prompts (at booking, by staff, by email) are working. It's the most direct measure of the second revenue stream
4	**Add-on revenue per recurring customer**	Add-on revenue ÷ active recurring customers	Upsell × recurring	Which add-ons to promote, drop, or reprice. Shows whether upsell is deepening existing customers or just riding on new ones
5	**Cash collected vs billed, and 60+ day overdue**	Confirmed receipts ÷ bills issued (trailing 4 weeks), and the 60+ bucket total from 4.3	Cash health	Whether to tighten payment terms or chase harder. Recurring revenue that isn't collected isn't revenue

Why these and not others. I left out total revenue on purpose. It moves when either the base or the upsell moves, so on its own it can't tell the founder which lever to pull. Metrics 1–2 and 3–4 separate those two effects. I also left out bookings volume, since for a recurring business it mostly follows active customers.

Judgment call: metric 2's reasons only work if cancellations are recorded with a reason in the CRM. If they aren't, the report shows the rate alone, and I'd suggest the client start capturing a reason.

Rule of the report: any metric that moves more than a set threshold vs its 8-week average gets one plain sentence underneath ("Attach rate fell from 34% to 26%; the drop is almost all in bookings made online"). Otherwise it's just the number. The founder should be able to read it in under a minute.

5. Three questions for the client

These are the three answers that would change the design the most. Everything else can be settled during the build.

1. **"When a customer pays you, what do they type as the reference, and is the bill number printed on the invoice they're paying from?"**
   - Fork: reconciliation tiers (4.4). If most payments carry the bill number, tier 1 auto-confirms the majority and the review queue is small; a spreadsheet review is enough. If references are blank or free text, almost everything lands in tiers 3–5, the review queue becomes daily work, and building a proper review screen moves from optional to necessary. It also tells me whether a card processor sits in between (A8), which would add a whole second data source.
2. **"How is the database mirror kept up to date: a nightly copy, or live replication? And could we get read access to the main database's change log?"**
   - Fork: change log (4.2) and job timing. A nightly copy confirms snapshot diffing and a fixed early-morning schedule. Live replication with log access would let me capture every change with before/after values instead of end-of-day state, and remove the "intra-day changes are invisible" limitation. That's worth the extra component *only if* the client actually needs intra-day history, e.g. for auditing staff edits to prices.
3. **"Is there one ID that follows a customer across bookings, bills and add-ons, and are add-ons stored as separate line items?"**
   - Fork: every report's data model. A stable customer ID and itemised add-ons mean all five jobs join cleanly. If customers are keyed by name or email, a deduplication step comes before everything else and match confidence drops across the board. If add-ons aren't itemised, founder metrics 3 and 4 can't be computed, and half the stated business model becomes unmeasurable from this data.

I chose these over questions about report formats or schedules because those are cheap to change after launch. These three decide the shape of the build.